BaitFL: Lightweight Trap-Based Free-Rider Detection and Mitigation in Federated Learning

Aashnan Rahman
Department of Computer Science and Engineering
Islamic University of Technology

***Abstract*—Free riders obtain the shared federated model without performing local training, while fabricated responses can resemble genuine updates. BaitFL uses undisclosed model challenges, update norms and tensor profiles to detect this behavior without training an auxiliary detector or accessing private training data. Statistical removal requires fresh evidence of the same component on two challenges and in two distinct audit cycles. Bounded review and rehabilitation limit temporary exclusion when evidence remains inconclusive. Multi-seed experiments cover MNIST and CIFAR-10, IID and non-IID data, four persistent fabrication strategies, and 30% and 40% free riders. All evaluated free riders are removed, with three honest removals. In separately timed 100-round executions, checking uses 0.58% of total computation time on MNIST and 0.89% on CIFAR-10. BaitFL is lightweight in server computation at the evaluated scale of 100 clients. Auditing nevertheless withholds about 11% of honest responses even in IID settings and requires individual-response visibility.**

***Index Terms*—federated learning, free-rider detection, hidden challenges, robust statistics**

I. Introduction

Federated learning allows clients to train a shared model while retaining their data locally. The server distributes a global model, clients update it using their own records, and returned updates are aggregated into the next model [1]. Keeping records local supports collaboration where data sharing is restricted, including healthcare. It does not eliminate privacy risks, since model updates can reveal information about training data [2], [3]. The collaboration also depends on clients doing useful work that the server cannot directly observe.

Free riders exploit this gap. Their aim is to obtain the shared model without paying the cost of local training [4], [5]. They may lack suitable data or computational resources, or simply choose to conserve them. Instead of training, they return an unchanged model, noise, or a response reconstructed from earlier models. Unlike poisoning attacks, these responses need not deliberately damage the global model. They can resemble ordinary updates while shifting the training burden to honest participants. When useful data and computation are withheld, the federation also loses contributions that could improve its model.

Detecting this omission is difficult under heterogeneous data. An honest client can produce a small or unusual update, while a fabricated one can follow the global training trajectory. Learned anomaly models add fitting cost, and audits based on local data or canary training introduce other requirements. A practical detector must consider both its server cost and the honest contributions lost through mistaken or prolonged exclusion.

We propose BaitFL, which challenges clients through the models they receive and checks whether their responses provide repeated evidence of free riding. Its contributions are:

- **Lightweight server-side detection.** Update norms and tensor profiles provide challenge-based evidence without an auxiliary detection model or access to private training records.
- **Confirmation using repeated evidence.** Statistical removal requires the same evidence component on both challenges and fresh support across two distinct complete audit cycles.
- **Bounded mitigation and rehabilitation.** Strong evidence survives ambiguous cycles, while unconfirmed quarantine expires and repeated clear observations reset statistical memory.
- **Evaluation across four behaviors.** A multi-seed study tests unchanged responses, random fabrication, history averaging and trajectory imitation on two datasets under IID and non-IID partitions and two attacker proportions.

II. Related Work

Lin *et al.* studied fabricated responses based on random weights and differences between received global models [4]. Their experiments showed that random weights can evade autoencoder detection, while model-difference updates can evade DAGMM. Their STD-DAGMM defense adds update standard deviation to an autoencoder and mixture model [6]. This improves separation in their experiments but requires fitting an auxiliary model. Fraboni *et al.* showed that stochastic disguises can let non-contributing clients approach the model obtained by honest participants [5]. Zhu *et al.* further studied convergence-aware fabricated trajectories [7]. Together, these results show why similarity to a trained update is insufficient evidence of local work.

FRAD combines learned detection with contribution estimates and reputation [8]. Historical information adds context, but requires reliable estimates of resources, communication and data quality alongside detector fitting. PASS instead asks clients to evaluate other participants' updates on their private datasets [9]. This supplies accuracy-based evidence and supports the authors' evaluated attacker-majority cases, at the cost of additional model transfers and client evaluation. Peer access to individual updates also broadens their exposure.

PASS uses perturbation and pruning to address this risk. Under heterogeneous data, low utility on another client's dataset need not imply that training was skipped.

FRIDA uses membership and property inference as training evidence [10]. Its canary variants introduce a deliberate training signal but require extra client training and server evaluation. Property-based variants avoid canary training while inferring potentially sensitive dataset characteristics. BaitFL changes the model initialization instead of supplying labeled examples and uses response summaries rather than inferred data properties. It still inspects individual responses, so it is incompatible with conventional secure aggregation that hides those responses [11].

Robust aggregation, including coordinatewise defenses and geometric median aggregation, limits the influence of harmful updates [12], [13]. Its objective differs from establishing whether a plausible-looking response involved training. BaitFL addresses the latter through controlled challenges and repeated confirmation, rather than treating every anomalous update as a free rider.

III. System and Threat Model

A trusted server coordinates a fixed population of N clients with persistent identities. Every active client responds in every round, without sampling, late arrivals or dropouts. Honest clients train the received model using the prescribed optimizer and may hold heterogeneous data. Quarantined clients still respond, but their responses are withheld from aggregation. Removal ends participation.

The server knows the model sent to client i in round t, $w_{t,i}^{\mathrm{send}}$, and its return $w_{t,i}^{\mathrm{ret}}$. It computes $\Delta_{t,i} = w_{t,i}^{\mathrm{ret}} - w_{t,i}^{\mathrm{send}}$ without accessing private records, local losses, training logs or resource reports. It does not know attacker identities, strategies or prevalence when making decisions. Individual responses remain visible to it.

We study persistent free riding without collusion, following Lin *et al.* [4]. Attackers perform no local optimization but can store models and make simple calculations. They use only their received-model history and local randomness, do not share responses or change identities, and do not adapt their strategies to audits. They are unaware of when traps are sent or how they are constructed. Partial training, intermittent free riding and targeted poisoning are outside this scope. An honest majority is assumed because screening compares peers and seeks a separated suspicious minority.

The four evaluated strategies return a fabricated model $\widehat{w}_{t,i}$:

- **FR1, unchanged response:** $\widehat{w}_{t,i} = w_{t,i}^{\mathrm{send}}$, regardless of whether it is a trap.
- **FR2, random fabrication:** perturb each floating model entry with independent uniform noise from $\mathcal{U}(-10^{-3}, 10^{-3})$.
- **FR3, history averaging:** average the most recent H received models, or all available models if fewer exist. The history includes the current model and any traps.
- **FR4, trajectory imitation:** add the mean $\overline{d}_{t,i}$ of up to five recent received-model differences, plus Gaussian noise:

$$\begin{aligned} \widehat{w}_{t,i} &= w_{t,i}^{\mathrm{send}} + \overline{d}_{t,i} + \eta_{t,i}, \\ \eta_{t,i} &= 0.1 \frac{\max(\|\overline{d}_{t,i}\|_2, 10^{-6})}{\max(\|z_{t,i}\|_2, 10^{-6})} z_{t,i}, \quad z_{t,i} \sim \mathcal{N}(0, I). \end{aligned} \tag{1}$$

Before a difference exists, FR4 instead adds independent Gaussian noise with standard deviation 10^{-4}. Traps enter its received history and therefore affect the estimated trajectory. Each run uses one strategy for all free riders.

IV. BaitFL Methodology

BaitFL alternates paired challenge cycles with ordinary training and sparse audits. Figure 1 shows this progression, and Algorithm 1 summarizes its decisions. A challenge changes the starting model while preserving the requested training task. Honest optimization responds through local data, whereas replay, arbitrary noise and recycled history need not respond in the same way. This gives behavioral evidence rather than proof of training.

A. Challenges and Response Summaries

After ten warmup rounds, each ten-round paired cycle freezes the current global model $w^{(c)}$ and constructs independent traps A and B. For floating tensor k,

$$\tau_{a,k} = w_k^{(c)} + 0.01 \max\{\mathrm{sd}(w_k^{(c)}), 10^{-4}\} \xi_{a,k}, \tag{2}$$

where $a \in \{A, B\}$ and $\xi_{a,k} \sim \mathcal{N}(0, I)$ independently. Both traps remain fixed throughout the cycle. Floating buffers are included and integer counters are unchanged.

Initially, clients are shuffled into at most ten groups, and each client receives each trap once. In later cycles, clients under review alternate between traps, while the remaining clients receive each trap once. For each client and trap, one response position is selected uniformly before responses are observed. Only those selected summaries enter the cycle decision, preventing retrospective selection of unusual responses. Incomplete paired coverage cannot authorize removal.

For each selected update Δ_i^a, the server records its norm $r_i^a = \|\Delta_i^a\|_2$ and normalized tensor profile

$$p_{i,k}^a = \frac{\|\Delta_{i,k}^a\|_2}{\sum_j \|\Delta_{i,j}^a\|_2}. \tag{3}$$

The profile is zero if the denominator is below 10^{-6}. It describes how update magnitude is distributed across tensors. Each client is compared with the other active clients receiving the same trap, including quarantined and unresolved peers. Let m_i^a, d_i^a be the median and median absolute deviation (MAD) of peer norms. For profile distance $\rho_i^a = \|p_i^a - \widetilde{p}_{-i}^a\|_2$, the reference $\widetilde{p}_{-i}^a$ is the coordinatewise peer median. Peer distances to that reference have median μ_i^a and MAD δ_i^a. Scores are

$$z_i^{n,a} = \frac{r_i^a - m_i^a}{d_i^a / \Phi^{-1}(0.75)}, \qquad z_i^{p,a} = \frac{\rho_i^a - \mu_i^a}{\delta_i^a / \Phi^{-1}(0.75)}. \tag{4}$$

Here Φ is the standard normal cumulative distribution function. Missing comparisons or zero MAD leave the score undefined. Exact send-back is checked separately through a zero update norm or a model fingerprint matching the immediately previous model sent to that client.

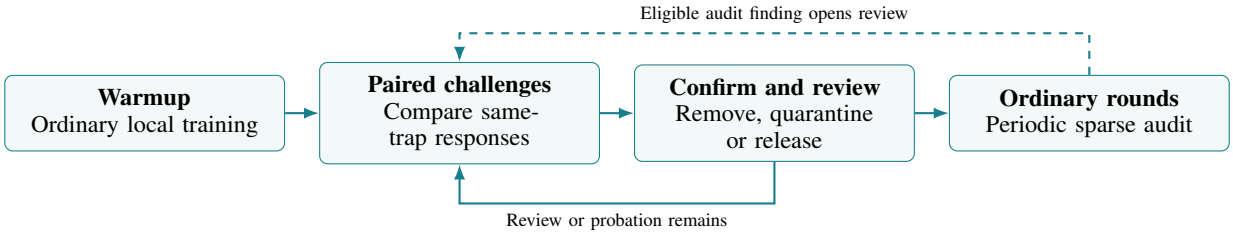


Fig. 1. BaitFL workflow.

B. Screening and Geometric Certification

Screening uses $D_i^a = z_i^{p,a} - z_i^{n,a}$ for each trap and their average. Each view has $\lceil \sqrt{n_v} \rceil$ histogram bins, where n_v is the number of valid paired observations. Otsu splitting nominates a minority above the boundary [14]. The lower side of the average-view split must contain a majority of all active clients. Joint client resampling across the three views gives 99% split intervals from 1,000 bootstrap replicates [15]. Scores above every upper boundary nominate suspicion, those below every lower boundary are clear, and the rest are unresolved. If a reliable split is unavailable, negative norm scores on both traps nominate suspicion. Neither negative is clear, and mixed or undefined scores are unresolved. The bootstrap intervals measure split stability, not false-removal risk.

A nomination alone cannot certify removal. Let $\mathcal{U}$ be the nominated cohort and $\mathcal{V}$ the remaining active population. With n active clients, statistical evidence requires

$$\left\lceil \frac{n}{\lceil \sqrt{n} \rceil} \right\rceil \leq |\mathcal{U}| < |\mathcal{V}|. \tag{5}$$

For a given raw signal, let Q_q denote an empirical quantile and $I_U = Q_{.75}(\mathcal{U}) - Q_{.25}(\mathcal{U})$. A low-norm certificate requires both

$$\begin{aligned} Q_{.25}(\mathcal{V}) - Q_{.75}(\mathcal{U}) > I_U, \\ \min(\mathcal{V}) - Q_{.75}(\mathcal{U}) > I_U, \end{aligned} \tag{6}$$

and an individual norm below the peer median. For high profile distances, write I_V for the peer IQR and $g = Q_{.25}(\mathcal{U}) - Q_{.75}(\mathcal{V})$. Require $g > I_U$, $g > 0$, and either

$$Q_{.25}(\mathcal{U}) - \max(\mathcal{V}) > I_U \quad \text{or} \quad \frac{I_U}{|\mathcal{U}| - 1} < \frac{I_V}{|\mathcal{V}| - 1}. \tag{7}$$

The individual profile distance must also exceed the peer median. Undefined measurements and insufficient support cannot certify a client. The profile rule permits a concentrated cohort after an interquartile gap, so persistent honest outliers can still pass it. These are geometric heuristics, not calibrated hypothesis tests.

C. Confirmation, Review and Rehabilitation

The same component must certify a client on both traps for a *strong* cycle outcome. An uncertified nomination is *unresolved* and does not automatically trigger quarantine. Separate norm and profile counters increment once per fresh complete cycle. Statistical removal requires a fresh certificate that raises its own component count to two. The cycles need not be consecutive, but norm and profile counts never combine.

Algorithm 1 BaitFL training and auditing

```
Initialize records, component counts and quiet clock σ = ⊥
Run ten ordinary warmup rounds
Schedule initial paired coverage
while the training horizon is not reached do
    if initial coverage, review or probation is active then
        Create two fixed traps and preselect response positions
        Run up to ten paired-cycle rounds, deferring final aggregation
        if paired coverage is complete then
            Screen peers and certify each component on both traps
            Update separate counts and consecutive send-back streaks
            Remove only on fresh confirmed evidence
            Apply quarantine, review expiry and rehabilitation
            σ ← current round if review/probation is empty, else ⊥
        end if
        Finish aggregation using the updated eligible set
    else if σ ≠ ⊥ and ten quiet rounds have passed then
        Send one fresh trap to all active clients
        F ← single-trap findings
        Open review for findings outside cooldown, without quarantine
        Decrease cooldown counters
        σ ← current round if F = ∅, else ⊥
    else
        Run one ordinary round and aggregate eligible responses
    end if
end while
```

Ambiguity preserves them and two consecutive clear cycles reset them. Exact send-back instead requires paired evidence in two consecutive complete cycles. Earliest removal under the study schedule is round 30.

Strong evidence starts quarantine and a two-cycle review budget. Fresh strong evidence renews it, while other completed cycles consume it. Expiry releases an unconfirmed client even if evidence remains in memory. Two consecutive clear outcomes rehabilitate a client, followed by one additional clear probation cycle. Unconfirmed release starts a cooldown of two sparse audits. Weak flags cannot renew exclusion. An unresolved client may contribute ordinary responses whenever it is not quarantined.

D. Sparse Audits and Aggregation

When review and probation are empty, ordinary training continues with an audit after ten quiet rounds. One fresh trap goes to all active clients. A single-trap certificate or exact send-back opens two cycles of review only if the client is outside cooldown. It cannot update statistical counts, quarantine

immediately or authorize removal.

The implemented quiet clock restarts only when the raw finding set $\mathcal{F}$ is empty, as shown in Algorithm 1. If all nonempty findings are blocked by cooldown, review remains empty and the unset clock can prevent subsequent audits. This pattern occurs in none of the 622 recorded audits. An eligible-set clock guard agrees on those traces but has not been used to recompute the results.

Let $\mathcal{A}_t$ contain ordinary-model recipients that are not quarantined or newly removed. Eligible updates receive equal weights:

$$w_{t+1} = w_t + \frac{1}{|\mathcal{A}_t|} \sum_{i \in \mathcal{A}_t} \Delta_{t,i}. \tag{8}$$

An empty set leaves the model unchanged. Trap responses never enter aggregation, and trap training replaces ordinary local training for that round. BaitFL and the no-defense comparison use the same client weighting. Settings are fixed across the reported grid rather than tuned separately for individual attacks or partitions.

V. Experimental Evaluation

A. Setup and Reporting

The study uses MNIST and CIFAR-10 [16], [17], IID and Dirichlet non-IID partitions with $\alpha = 0.5$, all four strategies, and 30% and 40% free riders. Each configuration has multi-seed evaluation. Federations contain 100 clients and run for 100 rounds. Honest clients train for three local SGD epochs with learning rate 0.01, momentum 0.9 and batch size 32. FR3 uses $H = 5$.

MNIST uses convolutional blocks with 16 and 32 channels and a 128-unit hidden classifier. CIFAR-10 uses 32, 64 and 128 channels with BatchNorm and a 256-unit classifier. Images are normalized without augmentation. Execution is on CPU with six PyTorch threads. Independent random streams control partitioning, identities, initialization, training and audits.

Detection rates count permanent removals, excluding temporary quarantine and unresolved status. Table I pools counts across attacks and seeds within each setting. Accuracy is the mean and sample standard deviation across seeds for each attack. Runs are the experimental units, since clients share a training trajectory. Pooled precision describes observed outcomes rather than uncertainty from independent client trials. Honest-response withholding counts trap and quarantine client-rounds once, relative to active honest client-rounds.

B. Detection and Honest-Client Impact

All 3,360 evaluated free riders are removed, with no false negatives. Three honest clients are removed among 6,240 honest client-cases, concentrated in two runs. Most executions finish detection at round 30, while the slowest reaches round 90. FR1 and FR2 are consistently removed at the earliest opportunity. The delayed cases involve history-based responses. Table I shows final mean accuracy between 98.83% and 99.02% on MNIST and between 74.51% and 77.84% on CIFAR-10.

All honest removals occur in MNIST non-IID at 40%. Two clients under FR3 receive repeated profile certificates and are removed at round 30. One under FR4 receives norm certificates in separated cycles and is removed at round 90. This setting has 480 correct and three incorrect removals, giving $480/483 = 99.38\%$ pooled precision. Repeated challenges can reinforce an error when legitimate data heterogeneity persistently resembles the geometric evidence used by the detector.

Temporary costs extend beyond those three errors. There are 32 honest client-cases with quarantine, totaling 620 client-rounds, although none remains quarantined at completion. Even IID cases with no honest removal or quarantine withhold roughly 11% of honest responses through the trap schedule. Non-IID FR3 at 40% withholds 12.86% on MNIST and 17.08% on CIFAR-10. These are lost contributions, regardless of server computation.

Unresolved status is also common after free riders have gone. The same non-IID FR3 setting ends with means of 37 unresolved honest clients on MNIST and 43 on CIFAR-10, out of 60 honest clients. The label alone does not prevent ordinary aggregation or renew quarantine. It records remaining evidence and is not a reliable final free-rider classification in these settings.

C. Utility Relative to No Defense

Completed seed-matched no-defense runs use the same partitions, training and equal-weight aggregation. Across available pairs, BaitFL's final accuracy differs by a mean of $+0.10$ percentage points on MNIST and $+0.60$ on CIFAR-10. These descriptive averages indicate comparable or slightly higher accuracy, rather than a consistent improvement in every setting. Baseline replication coverage is uneven across datasets.

Saved trajectories also permit illustrative time-to-target comparisons at 98% MNIST and 70% CIFAR-10 accuracy. Averaging seeds within each attack and then attacks equally, MNIST non-IID at 40% reaches the target in 16.48 minutes with BaitFL and 23.28 minutes without defense. The corresponding CIFAR-10 means are 48.02 and 58.32 minutes. Targets were chosen after examining the trajectories. These observations combine the benefit of removing free riders with the cost of withheld contributions. They do not isolate auditing overhead or establish a controlled timing advantage. An honest-only oracle and matched execution clocks are needed to separate these effects.

D. Instrumented Computation Cost

Separate 100-round executions compare BaitFL and a reimplementation of STD-DAGMM on non-IID FR3 at 40%, using the same client count, local epochs and CPU-thread setting. STD-DAGMM retains all clients and freshly fits an auxiliary model for five epochs per round. This fitting budget is a reproduction choice, not an author-specified schedule or evidence of convergence [4], [6].

Table II separates the checking stages, whose subtotals are included in total computation. BaitFL checking averages 0.215 seconds per round on MNIST and 0.791 on CIFAR-10,

TABLE I
DETECTION PERFORMANCE AND FINAL MODEL ACCURACY.

Dataset	Partition	FR (%)	Detection (%)		Final test accuracy (%)			
			Precision	Recall	FR1	FR2	FR3	FR4
MNIST	IID	30	100.00	100.00	99.01 ± 0.04	99.01 ± 0.04	99.02 ± 0.02	98.90 ± 0.07
		40	100.00	100.00	98.93 ± 0.06	98.94 ± 0.05	98.92 ± 0.07	98.83 ± 0.07
	Non-IID	30	100.00	100.00	98.92 ± 0.08	98.93 ± 0.06	98.93 ± 0.05	98.91 ± 0.05
		40	99.38	100.00	98.91 ± 0.07	98.91 ± 0.06	98.90 ± 0.05	98.88 ± 0.09
CIFAR-10	IID	30	100.00	100.00	77.84 ± 0.53	77.84 ± 0.31	77.76 ± 0.46	77.77 ± 0.37
		40	100.00	100.00	77.07 ± 0.55	77.13 ± 0.53	76.92 ± 0.52	76.97 ± 0.46
	Non-IID	30	100.00	100.00	75.94 ± 0.33	75.86 ± 0.65	75.77 ± 0.52	75.78 ± 0.36
		40	100.00	100.00	74.70 ± 1.22	74.88 ± 1.09	74.67 ± 0.64	74.51 ± 1.16

TABLE II
MEAN COMPUTATION TIME PER ROUND (SECONDS).

Stage	MNIST		CIFAR-10	
	BaitFL	STD	BaitFL	STD
Traps and scheduling	0.0018	0	0.018	0
Features/preparation	0.198	0.045	0.729	0.115
Auxiliary fitting	0	15.76	0	42.09
Scoring and decisions	0.016	0.335	0.043	0.827
Checking subtotal	0.215	16.14	0.791	43.03
All computation	36.92	52.26	88.56	120.05

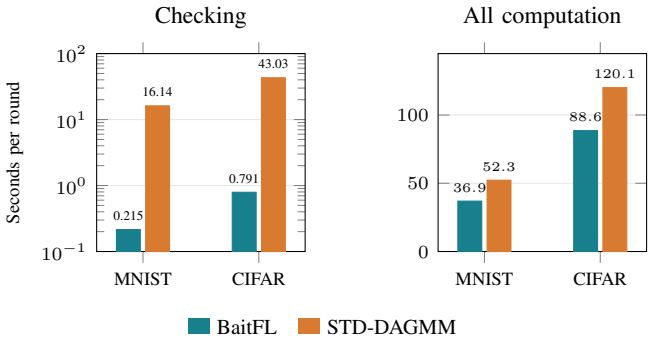


Fig. 2. Instrumented round times. BaitFL is the left bar of each pair. The checking panel uses a logarithmic scale.

respectively 0.58% and 0.89% of total computation including local training and global evaluation. These shares are not incremental overhead against no defense. STD-DAGMM fitting averages 15.76 and 42.09 seconds per round. Figure 2 shows the corresponding checking and total computation times. Overall computation is 29.35% lower on MNIST and 26.23% lower on CIFAR-10 in these particular executions.

The comparison measures two native workflows with different participation and runtime behavior. It does not rank detection effectiveness. The stored five-epoch STD-DAGMM flags identify none of the 4,000 free-rider responses in either execution, while flagging 2,200 honest responses on MNIST and 2,196 on CIFAR-10. These single-configuration roundwise flags are not matched to BaitFL's permanent-removal endpoint. A comparative detection benchmark requires validated baseline settings and common endpoints. Network transmission and device energy are not measured.

VI. COMPLEXITY AND LIMITATIONS

Let P be floating model coordinates, L floating tensors, N active clients, B bootstrap replicates and $K = \lceil \sqrt{N} \rceil$ histogram bins. Response features require $O(NP)$ work per round. Current leave-one-out comparisons cost $O(N^2(L + \log N))$ per decision, while bootstrap splitting costs $O(B(N + K))$. For T rounds, M paired cycles and A sparse audits, total detector work is bounded by

$$O\big(TNP + (M + A)\big[N^2(L + \log N) + B(N + K)\big]\big). \tag{9}$$

Working state is $O(P + NL + BN + BK)$, excluding ordinary training and research logs. Shared challenges and compact summaries avoid an auxiliary neural detector, but peer comparisons remain quadratic in population. The lightweight claim is therefore supported at 100 clients rather than established for arbitrarily large federations.

Challenges replace ordinary models in the existing exchange, preserving the nominal $O(NP)$ model-payload order per round. They still consume local training rounds whose outputs are withheld. Low server checking cost cannot establish low utility, network or energy cost.

The threat model excludes collusion, intermittent training and adaptive trap recognition. An attacker that recognizes a challenge or reproduces its response statistics may evade the detector. Honest non-IID clients can also produce persistent outliers, as the observed removals show. Two-trap and cross-cycle confirmation reduce reliance on an isolated observation but cannot resolve this fundamental ambiguity. Individual response visibility excludes conventional secure aggregation, and the fixed population does not test dropout or client sampling.

Experiments cover two datasets, two relatively high attacker shares and four specified non-adaptive strategies. They do not reproduce every published stochastic or convergence-aware attack. Detector settings were fixed after development without a separate held-out tuning study. Priorities for further evaluation include honest-only and low-prevalence conditions, passive-detector ablations, validated comparative detection, and controlled oracle timing. The cooldown-clock edge case also requires corrected-code reruns before broader implementation claims.

VII. Conclusion

BaitFL uses hidden model challenges, compact response features and repeated confirmation to detect persistent free riding without an auxiliary learned detector. All tested free riders are removed, with three honest removals concentrated in non-IID runs. Instrumented checking cost is small at 100 clients, while withheld honest contributions and persistent statistical ambiguity remain practical costs. The results support lightweight server computation under the stated threat model and motivate further tests of honest-only operation, lower attacker shares and adaptive behavior.

References

[1] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agüera y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, ser. Proceedings of Machine Learning Research, vol. 54. PMLR, 2017, pp. 1273–1282. [Online]. Available: https://proceedings.mlr.press/v54/mcmahan17a.html

[2] P. Kairouz, H. B. McMahan, B. Avent *et al.*, "Advances and open problems in federated learning," *Foundations and Trends in Machine Learning*, vol. 14, no. 1–2, pp. 1–210, 2021. [Online]. Available: https://arxiv.org/abs/1912.04977

[3] L. Melis, C. Song, E. D. Cristofaro, and V. Shmatikov, "Exploiting unintended feature leakage in collaborative learning," in *IEEE Symposium on Security and Privacy*, 2019, pp. 691–706. [Online]. Available: https://doi.org/10.1109/SP.2019.00029

[4] J. Lin, M. Du, and J. Liu, "Free-riders in federated learning: Attacks and defenses," arXiv preprint arXiv:1911.12560, 2019. [Online]. Available: https://arxiv.org/abs/1911.12560

[5] Y. Fraboni, R. Vidal, and M. Lorenzi, "Free-rider attacks on model aggregation in federated learning," in *Proceedings of the 24th International Conference on Artificial Intelligence and Statistics*, ser. Proceedings of Machine Learning Research, vol. 130. PMLR, 2021, pp. 1846–1854. [Online]. Available: https://proceedings.mlr.press/v130/fraboni21a.html

[6] B. Zong, Q. Song, M. R. Min, W. Cheng, C. Lumezanu, D. Cho, and H. Chen, "Deep autoencoding gaussian mixture model for unsupervised anomaly detection," in *International Conference on Learning Representations*, 2018. [Online]. Available: https://openreview.net/forum?id=BJJLHbb0-

[7] Z. Zhu, J. Shu, X. Zou, and X. Jia, "Advanced free-rider attacks in federated learning," in *NeurIPS Workshop on New Frontiers in Federated Learning*, 2021. [Online]. Available: https://neurips2021workshopfl.github.io/NFFL-2021/papers/2021/Zhu2021.pdf

[8] B. Wang, H. Li, X. Liu, and Y. Guo, "FRAD: Free-rider attacks detection mechanism for federated learning in AIoT," *IEEE Internet of Things Journal*, vol. 11, no. 3, pp. 4377–4388, 2024. [Online]. Available: https://doi.org/10.1109/JIOT.2023.3298606

[9] J. Wang, X. Chang, J. Mišić, V. B. Mišić, and Y. Wang, "PASS: A parameter audit-based secure and fair federated learning scheme against free-rider attack," *IEEE Internet of Things Journal*, vol. 11, no. 1, pp. 1374–1384, 2024. [Online]. Available: https://doi.org/10.1109/JIOT.2023.3288936

[10] P. G. Recasens, Á. Horváth, A. Gutierrez-Torre, J. Torres, J. L. Berral, and B. Pejó, "FRIDA: Free-rider detection using privacy attacks," *Journal of Information Security and Applications*, vol. 97, 2026, art. no. 104357. [Online]. Available: https://doi.org/10.1016/j.jisa.2025.104357

[11] K. Bonawitz, V. Ivanov, B. Kreuter, A. Marcedone, H. B. McMahan, S. Patel, D. Ramage, A. Segal, and K. Seth, "Practical secure aggregation for privacy-preserving machine learning," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, 2017, pp. 1175–1191. [Online]. Available: https://research.google/pubs/practical-secure-aggregation-for-privacy-preserving-machine-learning/

[12] D. Yin, Y. Chen, K. Ramchandran, and P. Bartlett, "Byzantine-robust distributed learning: Towards optimal statistical rates," in *Proceedings of the 35th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 80, 2018, pp. 5650–5659. [Online]. Available: https://proceedings.mlr.press/v80/yin18a.html

[13] K. Pillutla, S. M. Kakade, and Z. Harchaoui, "Robust aggregation for federated learning," *IEEE Transactions on Signal Processing*, vol. 70, pp. 1142–1154, 2022. [Online]. Available: https://doi.org/10.1109/TSP.2022.3153135

[14] N. Otsu, "A threshold selection method from gray-level histograms," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 9, no. 1, pp. 62–66, 1979. [Online]. Available: https://doi.org/10.1109/TSMC.1979.4310076

[15] B. Efron, "Bootstrap methods: Another look at the jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979. [Online]. Available: https://doi.org/10.1214/aos/1176344552

[16] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998. [Online]. Available: https://doi.org/10.1109/5.726791

[17] A. Krizhevsky, "Learning multiple layers of features from tiny images," University of Toronto, Tech. Rep., 2009. [Online]. Available: https://www.cs.toronto.edu/~kriz/cifar.html